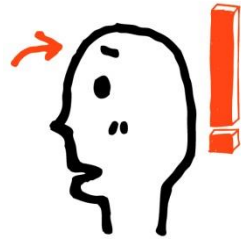




Android Java

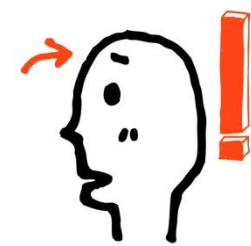
1장

이클립스로 시작하는 자바 첫걸음(Let's go JAVA)



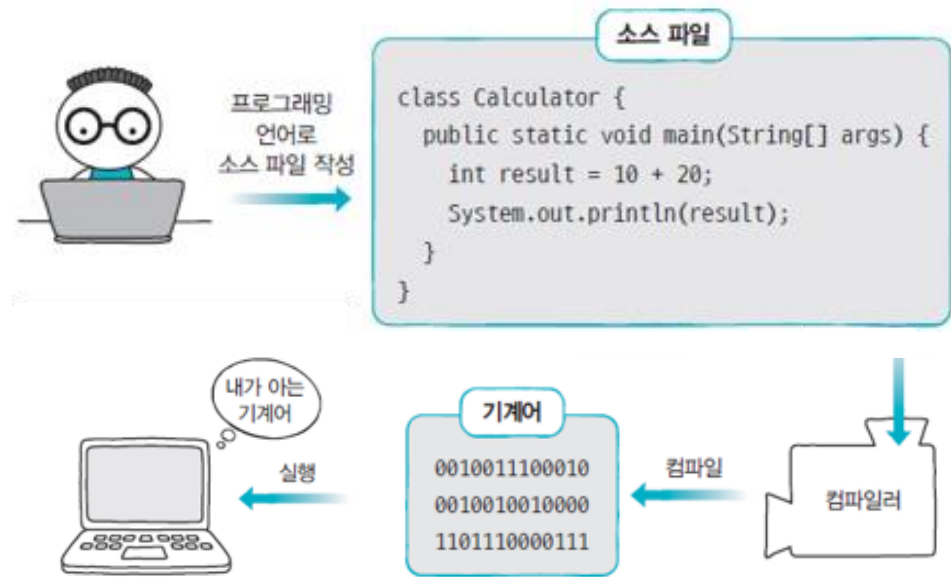
이클립스로 시작하는 자바 첫걸음

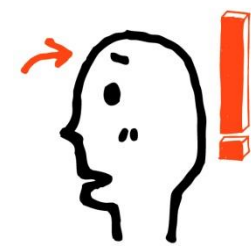
- 01** 개발자에게 사랑받는 언어, 자바와 친해지기
- 02** 자바에서 JVM의 역할은 무엇일까?
- 03** JDK 설치하고 환경 변수 설정하기
- 04** 프로그래밍의 시작, Hello World 예제를 만들어 보자
- 05** 실무에 적합한 개발 도구, 이클립스와 친해지자
- 06** 컴파일의 대상에서 제외되는 주석



프로그래밍 언어란?

- ❖ **프로그래밍 언어의 역할은?**
 - 사람과 컴퓨터의 대화 도움
 - 사람의 언어와 기계어 사이에서 다리와 같은 역할
 - 고급 언어와 저급 언어로 구분



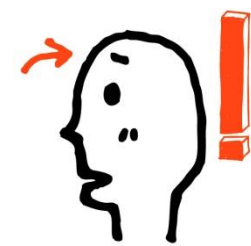


개발자에게 사랑받는 언어, 자바와 친해지기

- JAVA(자바) : 컴퓨터에서 작동하는 소프트웨어를 작성하는 프로그래밍 언어



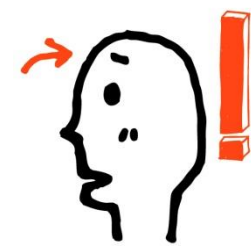
- Windows용 응용 프로그래밍에는 주로 C나 C++ 사용
- 실무의 서버용 응용 프로그램 및 안드로이드용 앱에는 JAVA, 코틀린이 가장 많이 사용



개발자에게 사랑받는 언어, 자바와 친해지기

“세상에는 **완벽한** 언어는 없다. 하지만 **훌륭한** 언어는 있다.”

- ▶ 1995년에 자바(Java)라는 객체 지향 언어가 정식으로 발표
- ▶ 전세계 개발자가 객체 지향 언어인 자바에 대해서 꾸준한 관심, 자바는 그들과 함께 성장
- ▶ 자바를 이용한 개발 영역이 매우 다양, 십 수 년 동안 꾸준히 사용
- ▶ 자생력이 있는 개발 언어
: 지금까지 자바는 부족한 부분을 보완하고 성능은 발전하면서 그 생명력을 키워가고 있음



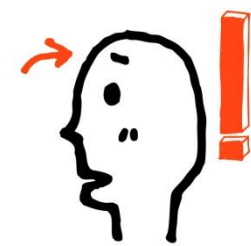
개발자에게 사랑받는 언어, 자바와 친해지기

▶ 자바 언어에 대한 소개

- ▶ 1990년대 썬 마이크로 시스템즈의 제임스 고슬링(James A. Gosling)이 개발
- ▶ 객체 지향 프로그래밍(Object Oriented Programming) : 개발 환경에 구애받지 않음

▶ 개발 목적에 따른 자바 언어 사용 환경(Platform, 이하 플랫폼) 분류

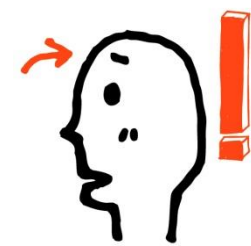
- ▶ Java SE(Java Standard Edition)
 - 자바 표준 플랫폼, JVM 운영 환경과 API2에 대한 개발 환경을 갖추고 있음
- ▶ Java EE(Java Enterprise Edition)
 - 서버 기능 개발을 위한 플랫폼, SE를 바탕으로 웹 애플리케이션 서버가 필요로 하는 기능 추가
- ▶ Java ME(Java Micro Edition)
 - 모바일 디바이스 혹은 셋탑 박스와 같은 임베디드 장비를 위한 개발에 사용되는 플랫폼



개발자에게 사랑받는 언어, 자바와 친해지기

▶ 자바의 6가지 특징

- ① 자바는 쉬운 문법을 제공한다.
- ② 자바는 독립적인 언어다.
- ③ 자바는 편리한 언어다.
- ④ 자바는 객체 지향 언어다.
- ⑤ 자바는 멀티 스레드 프로그래밍을 지원한다.
- ⑥ 자바를 지원하는 수많은 오픈 소스 프로젝트가 있다.



자바 개발 도구(JDK : Java Development Kit)

▶ 자바 개발 도구

▷ JDK 역할

자바 언어로 소프트웨어를 개발할 때 필요한 환경 및 도구를 제공하는 역할

▷ JDK 종류

Open JDK: <https://openjdk.java.net>

-개발, 학습용 및 상업용 모두 무료로 사용

Oracle JDK: <https://www.oracle.com>

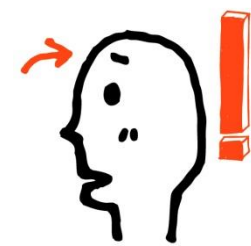
-개발, 학습용은 무료로 사용

-상업용 목적으로 사용할 경우 연간 사용료 지불

-장기 기술지원(LTS: Long Term Support) 및 업데이트 제공으로 안정적

▷ 학습용 JDK 선택

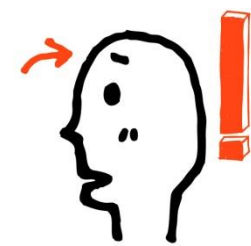
학습용은 모두 무료이므로 안정적인 **Oracle JDK**를 사용하는 것이 좋음



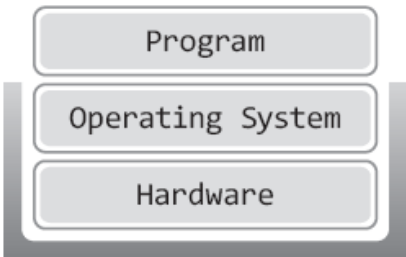
자바 개발 도구(JDK : Java Development Kit)

표 1-1 JAVA 버전의 변천사

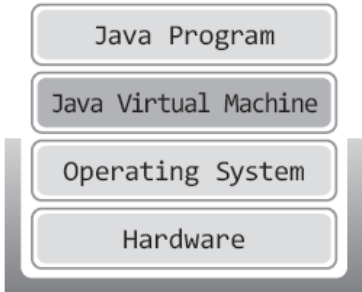
JAVA 버전	발표 일자	지원 만료일	비고
JDK 1.0	1996년 1월		JAVA 1이라고도 부름.
JDK 1.1	1997년 2월		JAVABeans, JDBC 등이 추가됨.
J2SE 1.2	1998년 12월		JAVA 2라고도 부르며, JDK 대신에 버전 이름을 J2SE(JAVA 2 Platform, Standard Edition)라고 명명함.
J2SE 1.3	2000년 5월		HotSpot JVM이 포함됨.
J2SE 1.4	2002년 2월		IPV6를 지원함.
J2SE 5.0	2004년 9월		내부 버전은 1.5이지만 이때부터 5.0 버전이라 명명함.
JAVA SE 6	2006년 12월	2018년 12월	J2SE 대신에 JAVA SE라 명명하고 버전에서 .0 숫자를 떼어냄.
JAVA SE 7	2011년 7월	2022년 7월	프로그래밍 과정을 단순화하고 JVM의 성능을 대폭 향상함.
JAVA SE 8	2014년 3월	2025년 3월	람다식(lambda expression)을 지원하고, 32bit를 지원하는 마지막 버전임.
JAVA SE 9	2017년 9월	2018년 3월	non-LTS. 이후로는 64bit만 지원함.
JAVA SE 10	2018년 3월	2018년 9월	non-LTS. 로컬 형식 추론, 가비지 수집과 관련해 개선됨.
JAVA SE 11	2018년 9월	2026년 9월	LTS(Long Term Support) 버전임.
JAVA SE 12	2019년 3월	2019년 9월	non-LTS.



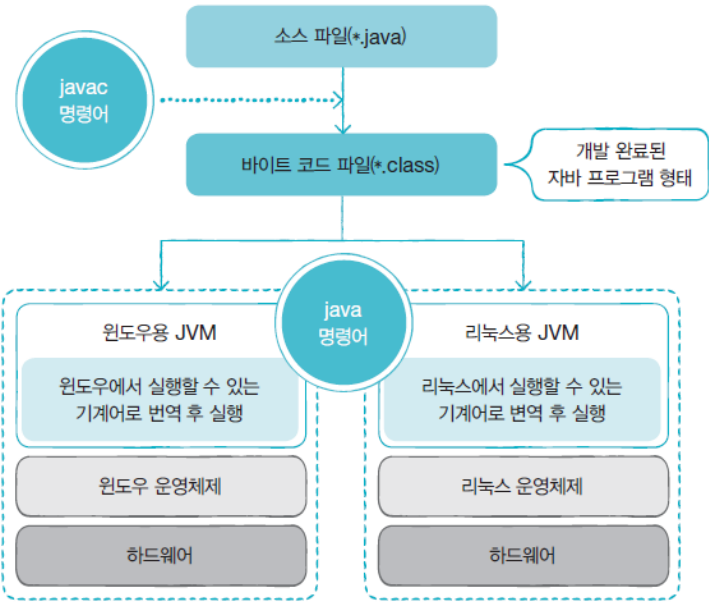
자바에서 JVM의 역할은 무엇일까?



일반적인 프로그램의 실행구조



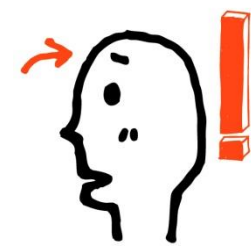
자바 프로그램의 실행구조



운영체제에 독립적인 자바 프로그램

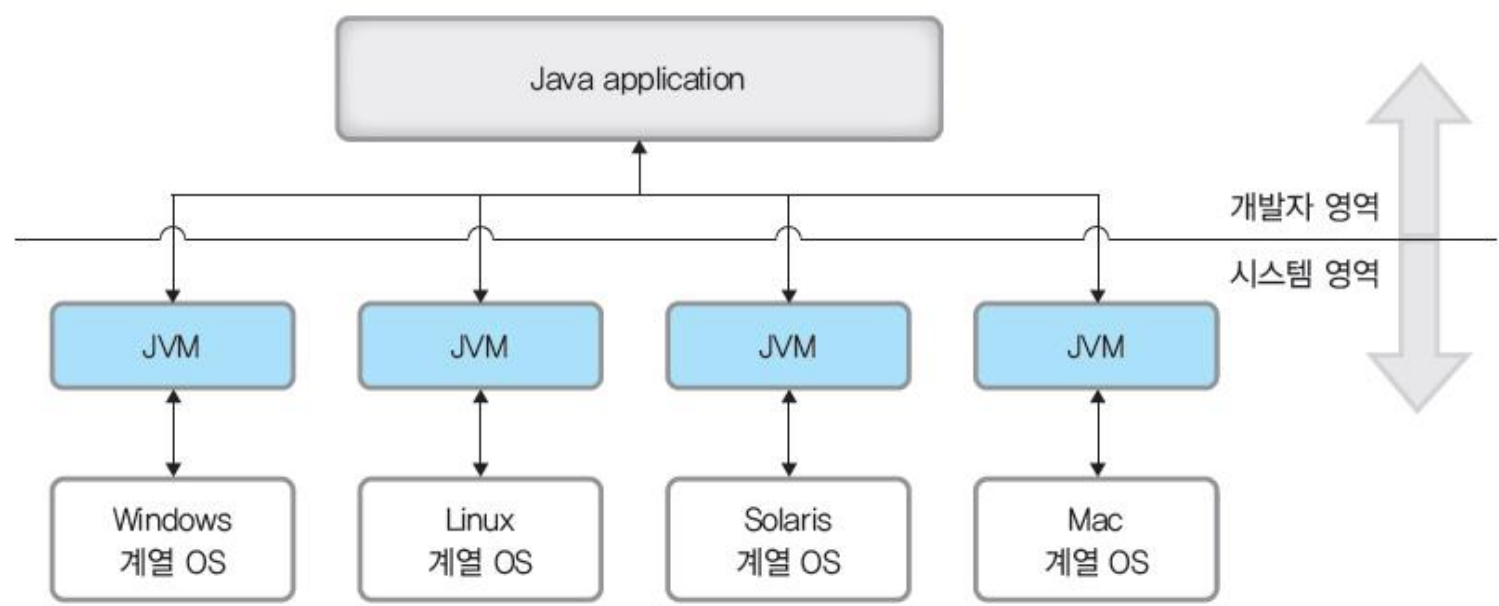
자바 프로그램은 운영체제에 독립적!

자바 가상머신은 운영체제에 의존적!

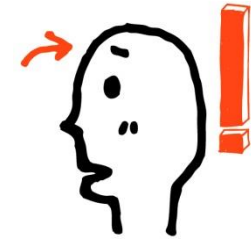


자바에서 JVM의 역할은 무엇일까?

- ▶ JVM이란? 자바 프로그램을 실행시키는 주체
- ▷ JVM이 OS와 프로그램의 매개체 역할을 하고 애플리케이션은 JVM 위에서 동작하도록 설계

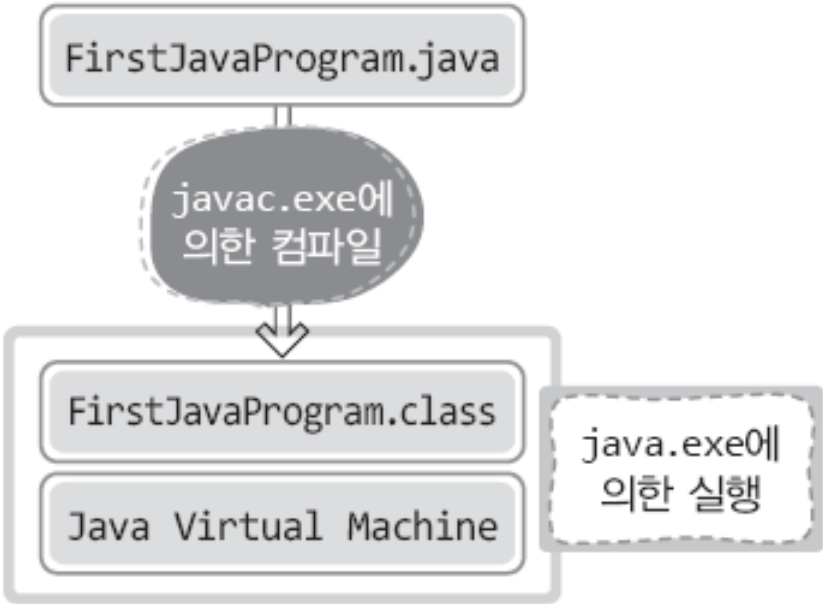


△ 그림 1-1 JVM과 OS의 종류

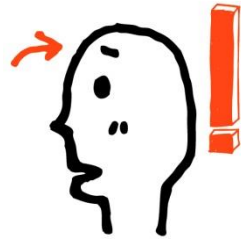


자바에서 JVM의 역할은 무엇일까?

▶ 자바 컴파일러와 자바 바이트 코드



자바 컴파일러는 자바 가상머신이 인식할 수 있는 자바 바이트 코드를 생성해 낸다!

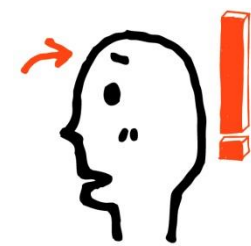


자바에서 JVM의 역할은 무엇일까?

▶ 구조로 알아보는 JVM의 동작 원리

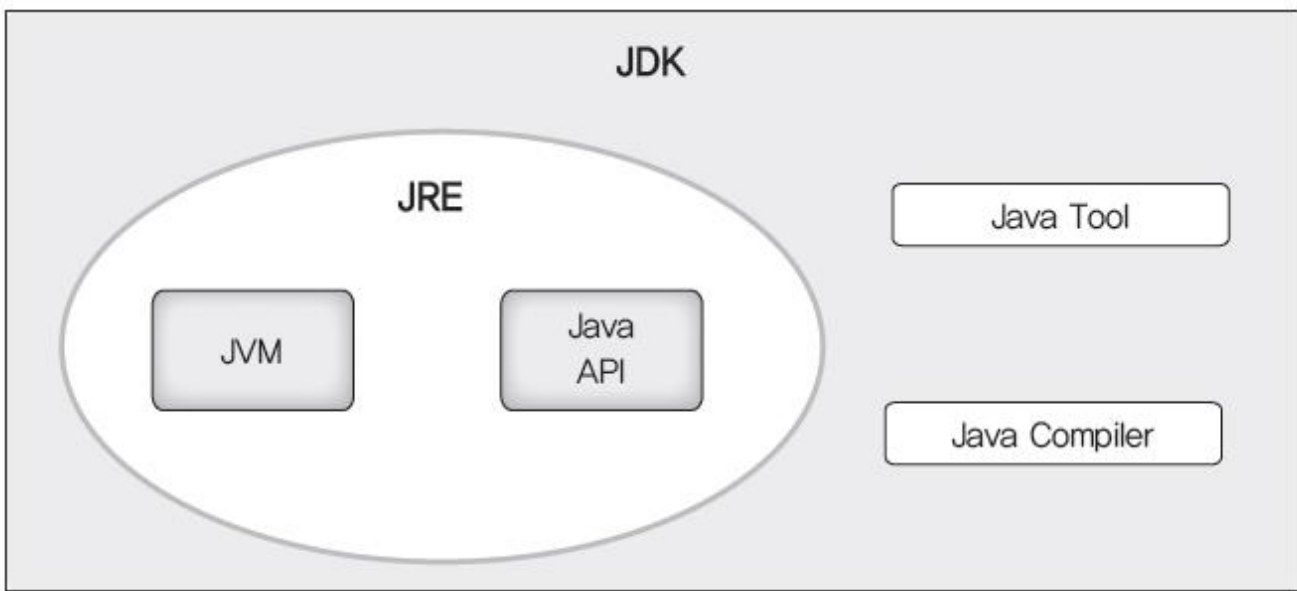
① 클래스 파일

- ▶ 개발자가 만드는 혹은 이미 만들어진 프로그램
- ▶ 개발할 자바 프로그램은 파일 확장자가 '.java'
- ▶ '.java' 파일이 자바 컴파일러에서 컴파일 과정을 거치면 '.class' 파일이 생성
- ▶ 컴파일 과정을 거쳐 생성된 클래스 파일은 JVM에서 실행 가능
- ▶ 클래스 파일이 서로 유기적으로 동작하면서 프로그램은 자기의 기능을 수행

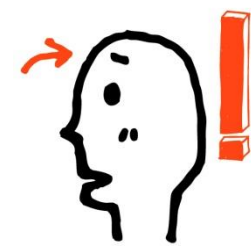


JDK 설치하고 환경 변수 설정하기

- ▶ **JRE** 자바 애플리케이션을 실행하기 위한 최소의 실행 환경, JVM과 기본 API들로 구성
- ▶ **JDK** JRE에서 제공하는 실행 환경뿐만 아니라 자바 개발에 필요한 여러 가지 명령어가 포함
→ 설치 권장

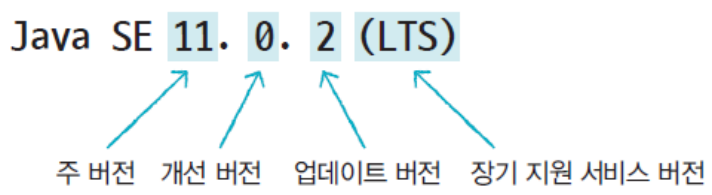


△ 그림 1-3 JDK vs JRE



JDK 설치하고 환경 변수 설정하기

JDK 버전 체계

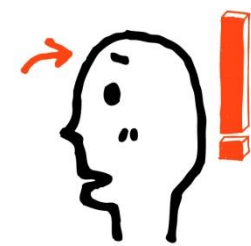


JDK 버전	설명
Java SE 12	주 버전이 12
Java SE 11.0.2 (LTS)	주 버전이 11이고, 수정이 2번 되었음 장기 지원 서비스를 제공받을 수 있는 버전
Java SE 8u202	주 버전이 8이고, 수정이 202번 되었음

설치하기: <https://www.oracle.com>

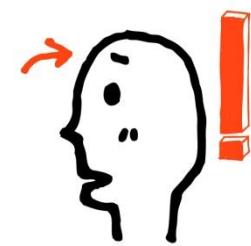
- [Downloads] 클릭
- [Developer Downloads] > Java
- [Java (JDK) for Developers]





JDK 설치하고 환경 변수 설정하기

- 설치할 PC 사양에 맞는 설치 파일을 다운로드
 - 64비트 윈도우 운영체제의 경우:
 - JDK 11 이후 버전: `jdk-xx.x.x_windows-x64_bin.exe`
 - JDK 8 버전: `jdk-8uxxx-windows-x64.exe`
 - 32비트 윈도우 운영체제의 경우:
 - JDK 8 버전: `jdk-8uxxx-windows-i586.exe`
- 설치 방법:
 - 기본 설치
- 설치된 경로
 - JDK 11 이후 버전: `C:\Program Files\Java\jdk -xx.x.x`
 - JDK 8 버전: `C:\Program Files\Java\jdk1.8.0_xxx`

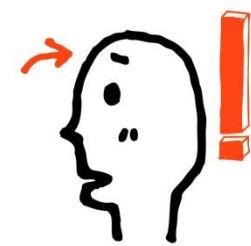


JDK 설치하고 환경 변수 설정하기

■ JAVA_HOME

- JDK가 설치된 폴더
- JDK 위치를 찾을 때 JAVA_HOME 환경 변수 이용 필요한 경우 있음
- JAVA_HOME 환경 변수 만들고 JDK 설치 폴더 등록
 - ① 윈도우 [시스템 속성] 대화상자 이용

윈도우 버전	[시스템 속성] 대화상자 실행 방법
Windows 7	시작 → 제어판 → 시스템 및 보안 → 시스템 → 고급 시스템 설정
Windows 8	화면 오른쪽 아래로 마우스 포인터 옮김 → 설정 → 제어판 → Windows 7과 동일
Windows 10	검색 → '제어판' 입력 후 선택 → Windows 7과 동일



JDK 설치하고 환경 변수 설정하기

② '시스템 속성' 창에서 [고급] 탭 클릭 후 아래쪽의 [환경 변수] 클릭

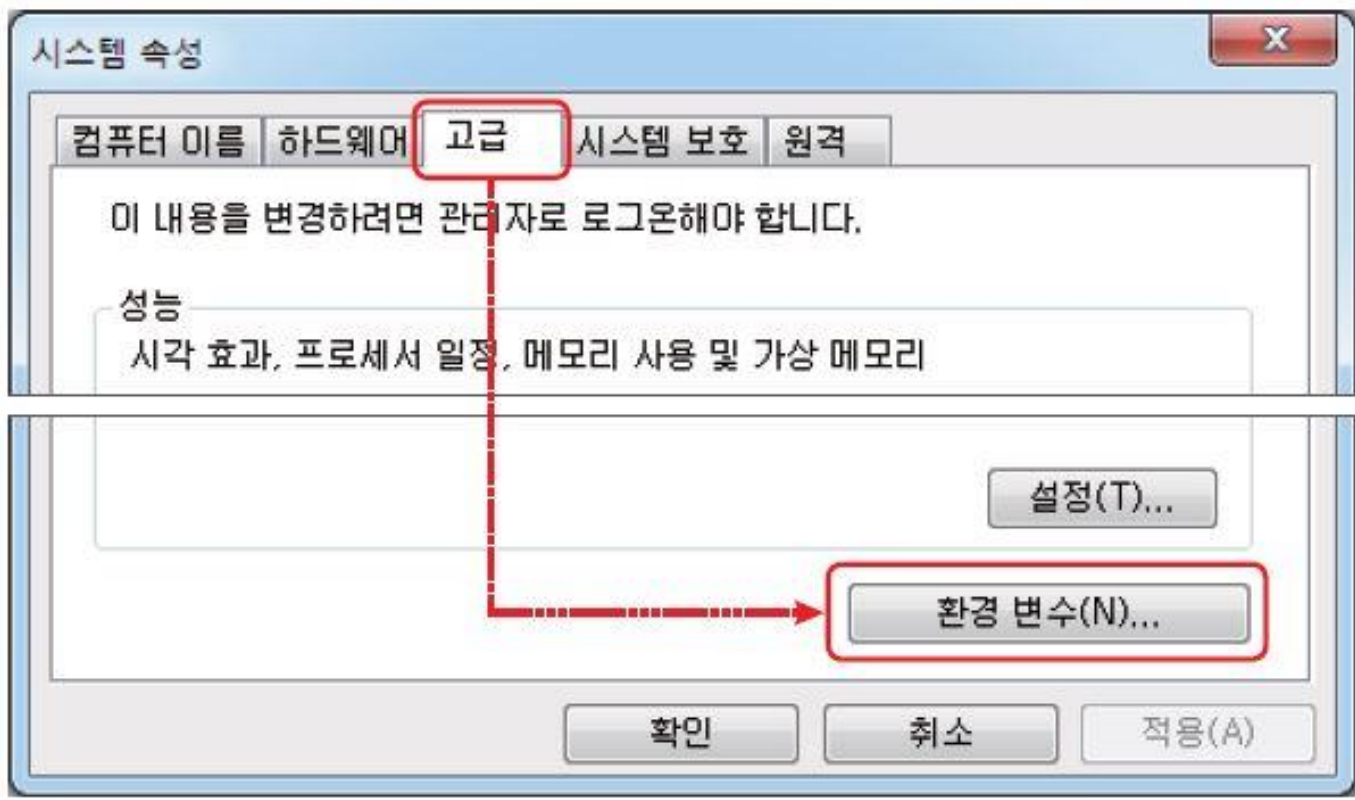
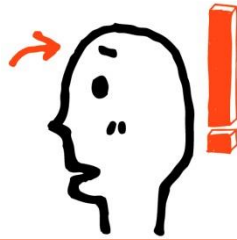


그림 1-15 환경 변수 설정



JDK 설치하고 환경 변수 설정하기

- ③ '환경 변수' 창에서 아래쪽 '시스템 변수'의 [새로 만들기] 클릭

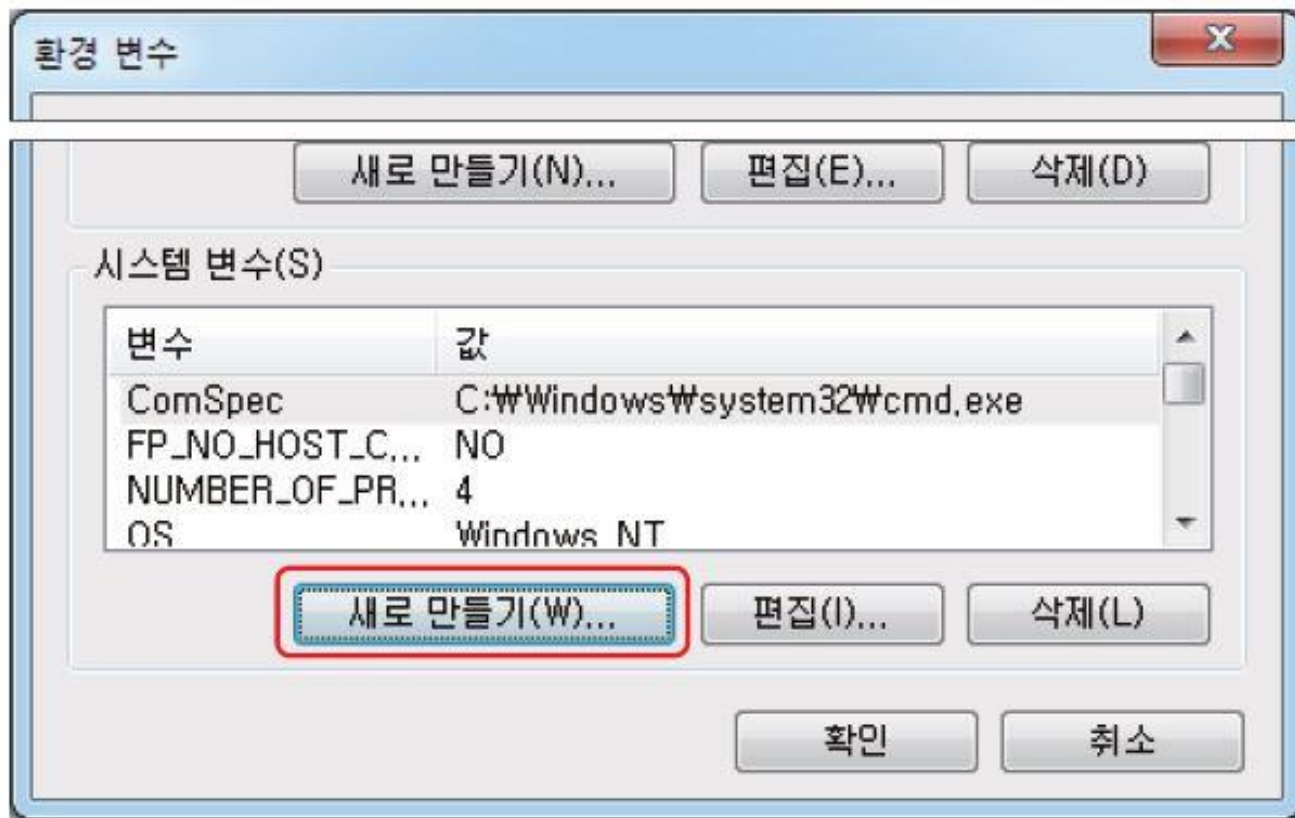
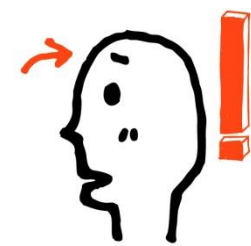


그림 1-16 JAVA_HOME 환경 변수 등록 1



JDK 설치하고 환경 변수 설정하기

- ④ '새 시스템 변수' 창에서 변수 이름에 'JAVA_HOME'을, 변수 값에 JDK가 설치된 경로 (필자의 경우 C:\Program Files\Java\jdk1.8.0_40)를 입력하고 [확인] 클릭

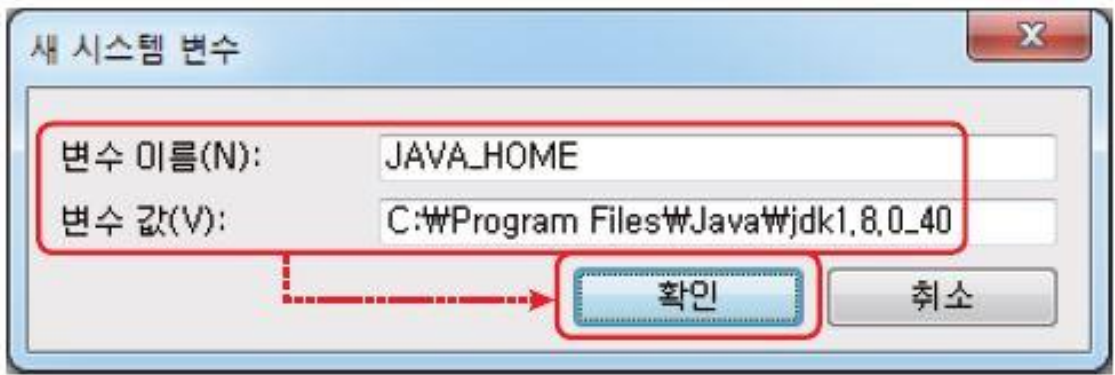
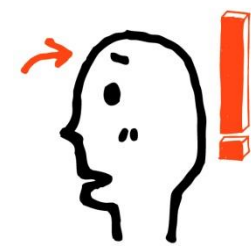


그림 1-17 JAVA_HOME 환경 변수 등록 2



JDK 설치하고 환경 변수 설정하기

⑤ JAVA_HOME 환경 변수 등록 확인

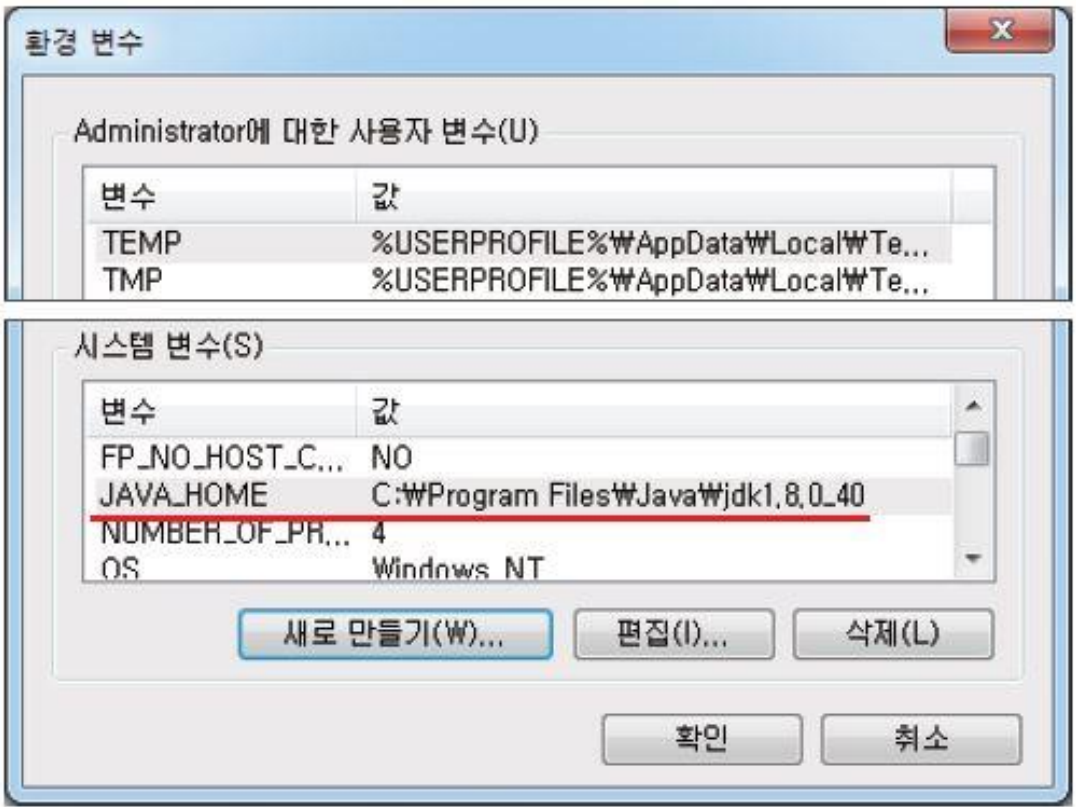
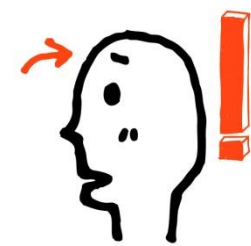


그림 1-18 등록된 환경 변수 확인



JDK 설치하고 환경 변수 설정하기

- Path 환경 변수에 JDK 실행 파일 폴더의 경로 추가

⑥ '시스템 변수' 중 'Path' 선택하고 [편집] 클릭

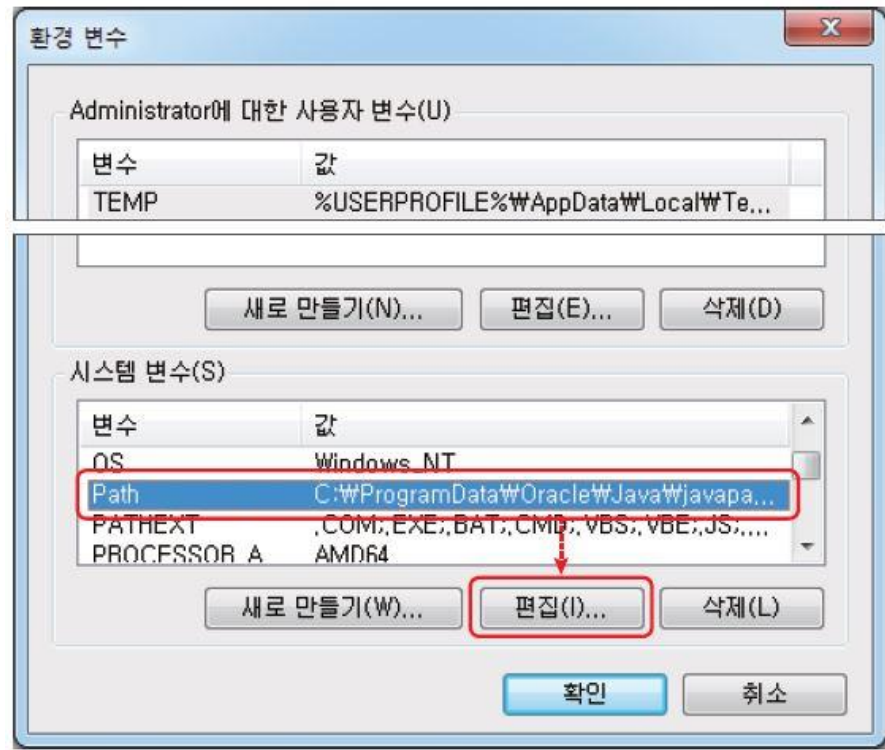
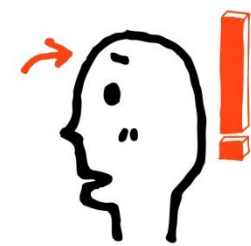
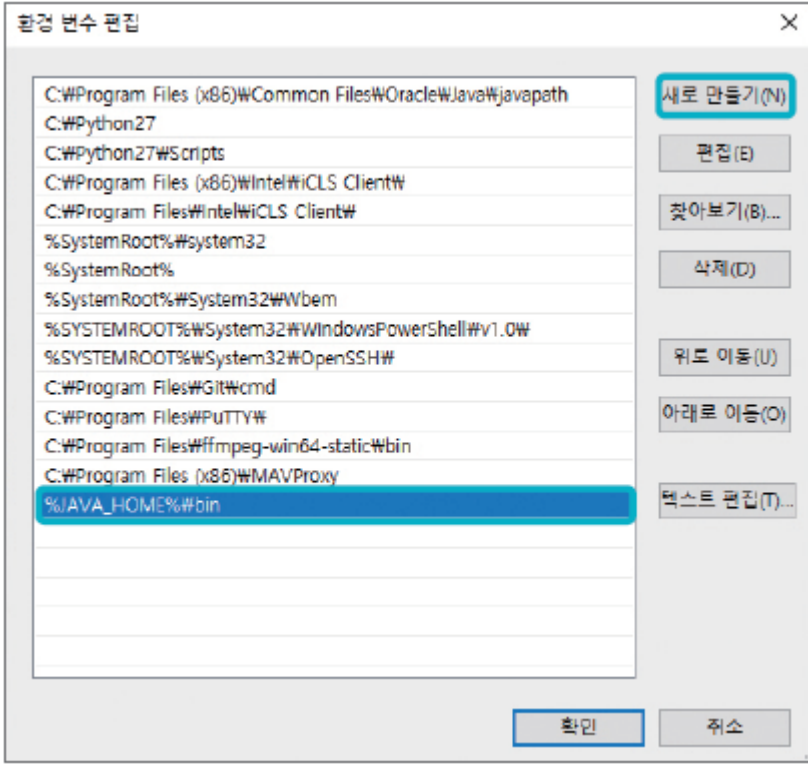


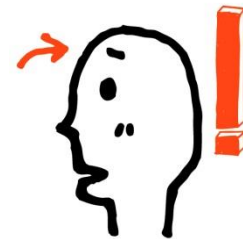
그림 1-19 Path 환경 변수에 경로 추가 1



JDK 설치하고 환경 변수 설정하기

⑦ [새로 만들기] - %JAVA_HOME%\bin 입력





JDK 설치하고 환경 변수 설정하기

▶ 편리한 개발 환경을 위한 환경 변수 설정

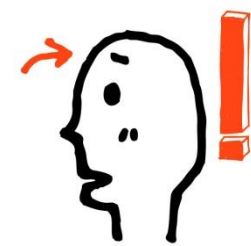
```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Benjamin>cd %

C:\>javac -version
javac 1.7.0_03

C:\>java -version
java version "1.7.0_03"
Java(TM) SE Runtime Environment (build 1.7.0_03-b05)
Java HotSpot(TM) 64-Bit Server VM (build 22.1-b02, mixed mode)
```

- ① 자바 환경 변수가 정상적으로 설정되었는지 확인하기 위해서 윈도우 커맨드 창을 실행
javac -version과 **java -version** 명령어를 실행
그림과 같은 화면이 출력된다면 성공적으로 Path 변수가 설정된 것



JDK 설치하고 환경 변수 설정하기

▶ Path 설정의 확인

```
C:\WINDOWS\system32\cmd.exe
C:\>java
Usage: java [-options] class [args...]
        <to execute a class>
    or  java [-options] -jar jarfile [args...]
        <to execute a jar file>

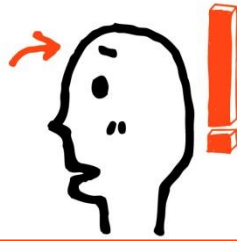
where options include:
    -client      to select the "client" VM
    -server      to select the "server" VM
    -hotspot     is a synonym for the "client" VM [depr
The default VM is client.
```

자바 런처의 실행

```
C:\WINDOWS\system32\cmd.exe
C:\>javac
Usage: javac <options> <source files>
where possible options include:
    -g           Generate all debugging info
    -g:none      Generate no debugging info
    -g:{lines,vars,source}
                  Generate only some debugging
    -nowarn      Generate no warnings
    -verbose     Output messages about what t
    -deprecation Output source locations wher
sed
    -classpath <path> Specify where to find user
```

자바 컴파일러의 실행

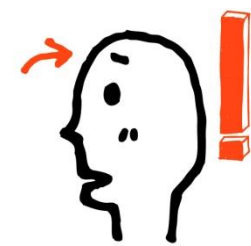
path가 제대로 설정되었다면
실행 결과에서 보이듯이 java,
그리고 javac가 실행되어야 한다.



프로그래밍의 시작, HelloJava 예제를 만들어 보자

실습 1-1 처음으로 만드는 JAVA 프로그램

```
01 public class HelloJava
02 {
03     public static void main (String[] args)
04     {
05         System.out.println("안녕? Java ~"); ----- System.out.println()은 화면에 내용을 출력한다.
06     }
07 }
```



프로그래밍의 시작, HelloJava 예제를 만들어 보자

JAVAC HelloJava.java → 소스코드 컴파일
DIR

```
관리자: 명령 프롬프트

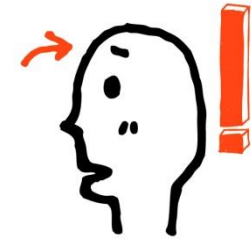
C:\CookJava\Chapter01\HelloJava> JAVAC HelloJava.java

C:\CookJava\Chapter01\HelloJava> DIR
C 드라이브의 볼륨에는 이름이 없습니다.
볼륨 일련 번호: 9EC5-96A6

C:\CookJava\Chapter01\HelloJava 디렉터리

2015-08-15   오후 10:43   <DIR>          .
2015-08-15   오후 10:43   <DIR>          ..
2015-08-15   오후 10:43               427 HelloJava.class
2015-08-04   오후 10:56               119 HelloJava.java
                2개 파일                546 바이트
                2개 디렉터리  51,866,730,496 바이트 남음

C:\CookJava\Chapter01\HelloJava>
```



프로그래밍의 시작, HelloJava 예제를 만들어 보자

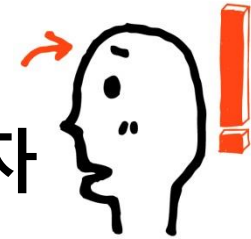
JAVA HelloJava → 바이트코드 실행

```
관리자: 명령 프롬프트

C:\CookJava\Chapter01\HelloJava> JAVA HelloJava
안녕? Java ~~~

C:\CookJava\Chapter01\HelloJava>
```

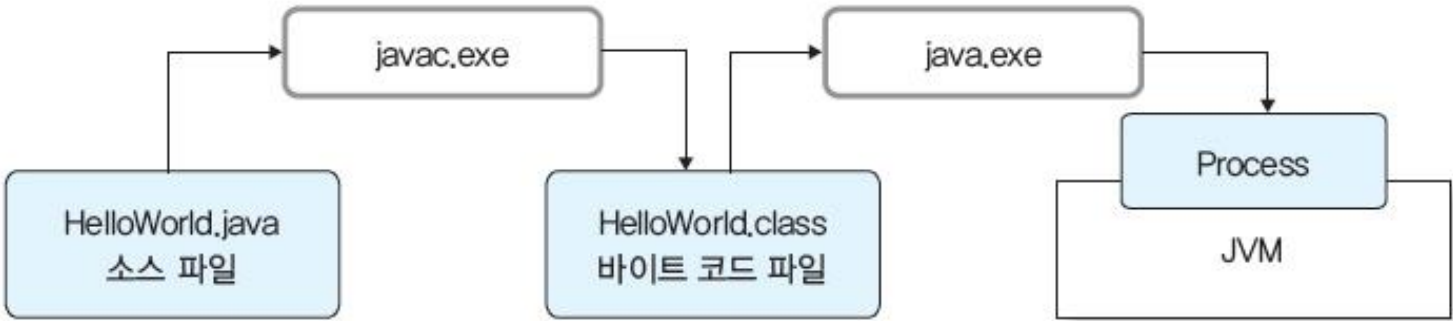
그림 1-29 실행 및 결과 확인



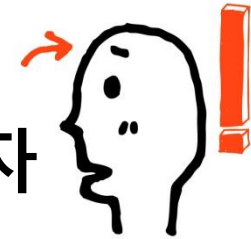
프로그래밍의 시작, Hello World 예제를 만들어 보자

▶ HelloWorld.java 실행하기

- ▶ **컴파일이란?** 사람이 읽을 수 있는 형태의 코드(java 파일)를 컴퓨터가 읽을 수 있는 기계어 형태의 코드(class 파일)로 변환하는 것
- ▶ 자바에서는 컴파일 과정을 거치면 JVM이 읽을 수 있는 바이너리 코드로 변환
- ▶ 변환된 '.class' 파일은 바이너리 파일 혹은 바이트 코드, 클래스 파일 등의 이름으로 불림



△ 그림 1-15 HelloWorld 컴파일 및 실행 과정



프로그래밍의 시작, Hello World 예제를 만들어 보자

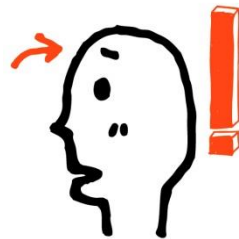
■ HelloJava 프로그램 뜯어보기

■ HelloJava 분석

실습 1~2 처음으로 만드는 JAVA 프로그램(다시 보기)

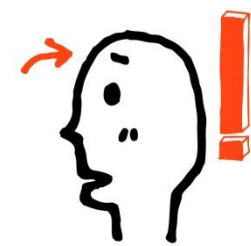
```
01 public class HelloJava
02 {
03     public static void main (String[] args)
04     {
05         System.out.println("안녕? Java ~~");
06     }
07 }
```

- 1행의 `public class HelloJava`에서 `public class`는 예약어로 일단은 항상 동일하게 쓰는 것. `HelloJava`는 클래스 이름, 파일 이름인 `HelloJava.java`에서 `.java` 제외한 것과 동일해야 함(클래스는 JAVA 프로그램을 구성하는 가장 기본 단위). 1행의 `HelloJava` 클래스는 2행의 시작 중괄호(`{`)부터 7행의 종료 중괄호(`}`)까지 본체임.
- 3행의 `public static void main (String[ ] args)`는 JAVA 프로그램의 핵심 명령어를 코딩할 메인 메소드
- 3행의 메소드는 4행의 시작 중괄호(`{`)부터 6행의 종료 중괄호(`}`)까지 범위이며 이 안에 JAVA 프로그램 작성. 즉 이 예제는 5행의 한 줄만 코딩되어 있는 것으로 '안녕? Java ~~'를 출력



키워드로 끝내는 핵심 포인트

- **기계어**: 컴퓨터(운영체제)가 이해하고 실행할 수 있는 0과 1로 이루어진 코드를 말합니다.
- **프로그래밍 언어**: 사람이 기계를 이해하는 것은 매우 어렵기 때문에 사람의 언어와 기계어의 다리 역할을 합니다. 종류로는 C, C++, 자바^{Java}, 파이썬^{Python} 등이 있습니다.
- **소스 파일**: 프로그래밍 언어로 작성된 파일을 말합니다.
- **컴파일**: 소스 파일을 기계어로 번역하는 것을 말합니다. 이 역할을 담당하는 소프트웨어를 컴파일러라고 합니다.
- **JDK**: 자바 개발 도구^{Java Development Kit}의 줄임말로, 자바로 프로그램을 개발할 수 있는 실행 환경(JVM)과 개발 도구(컴파일러) 등을 제공합니다.
- **환경 변수**: 운영체제가 실행하는 데 필요한 정보를 제공해주는 변수를 말합니다. JDK를 설치한 후 명령 라인(명령 프롬프트, 터미널)에서 컴파일러(javac)와 실행(java) 명령어를 사용하려면 JAVA_HOME 환경 변수를 등록하고 Path 환경 변수를 수정하는 것이 좋습니다.



실무에 적합한 개발 도구, 이클립스와 친해지자

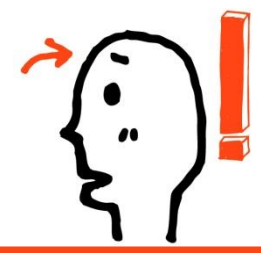
- 실무에서 JAVA를 개발할 때는 전용 툴(tool)을 사용, 다양한 JAVA 개발 툴 중에서도 이클립스(Eclipse)가 가장 많이 사용
- 이클립스 : JAVA 프로그램을 작성하도록 도와주는 편리하고 강력한 기능 많이 가지고 있으며 무료. 사용법도 어렵지 않아서 대부분의 JAVA 프로그래머가 이용하는 개발 툴



아이스박스(메모장)

고급 냉장고(이클립스)

그림 2-1 메모장과 이클립스의 비교



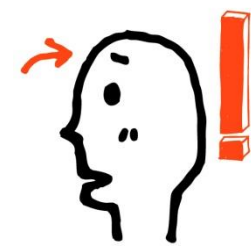
실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 이클립스의 특징(사전에 JDK 설치)

- ▶ 오픈 소스 기반에 자바 기반의 확장 가능한 개발 플랫폼
- ▶ 기본적으로 자바 프로그램을 개발하는 환경
- ▶ Plug-in 방식을 통해 다양한 자바 프로그램을 개발할 수 있도록 지원
- ▶ Auto-Compiling 기능을 통해 프로그램 작성 시 실시간으로 컴파일을 하여 코드 작성시 바로 문법적인 오류를 확인



좋은 편집 툴이란
개발자에게 보다
정확한 코딩을 유
도하도록 도와주
는 것!



실무에 적합한 개발 도구, 이클립스와 친해지자

■ 이클립스 설치

- 설치 파일 다운로드(실습)
- Eclipse IDE 2019-03 버전 이후부터는 기본적으로 64비트 설치용 인스톨러 파일 제공



Get **Eclipse IDE 2019-03**
Install your favorite desktop IDE packages.

[Download 64 bit](#)

[Download Packages](#) | [Need Help?](#)

[Download](#)

Download from: Philippines - RISE ([http](#))

File: `eclipse-inst-win64.exe` `SHA-512`

[>> Select Another Mirror](#)

- ZIP 또는 32비트 버전도 다운로드 가능



Get **Eclipse IDE 2019-03**
Install your favorite desktop IDE packages.

[Download 64 bit](#)

[Download Packages](#) | [Need Help?](#)

MORE DOWNLOADS

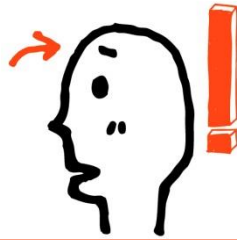
- Other builds
- [Eclipse 2018-12 \(4.10\)](#)
- [Eclipse 2018-09 \(4.9\)](#)
- Eclipse Photon (4.8)
- Eclipse Oxygen (4.7)
- Eclipse Neon (4.6)
- Eclipse Mars (4.5)
- Eclipse Luna (4.4)
- Eclipse Kepler (4.3)
- Older Versions

Eclipse IDE for Java EE Developers

339 MB 568,036 DOWNLOADS

Tools for Java developers creating Java EE and Web applications, including a Java IDE, tools for Java EE, JPA, JSF, Mylyn, EGit and others.

Windows [32-bit](#) [64-bit](#)
Mac Cocoa [64-bit](#)
Linux [32-bit](#) [64-bit](#)



실무에 적합한 개발 도구, 이클립스와 친해지자

- 'Select a workspace' 창에서 C:\CookJava\라고 직접 써서 변경 후 [OK] 클릭

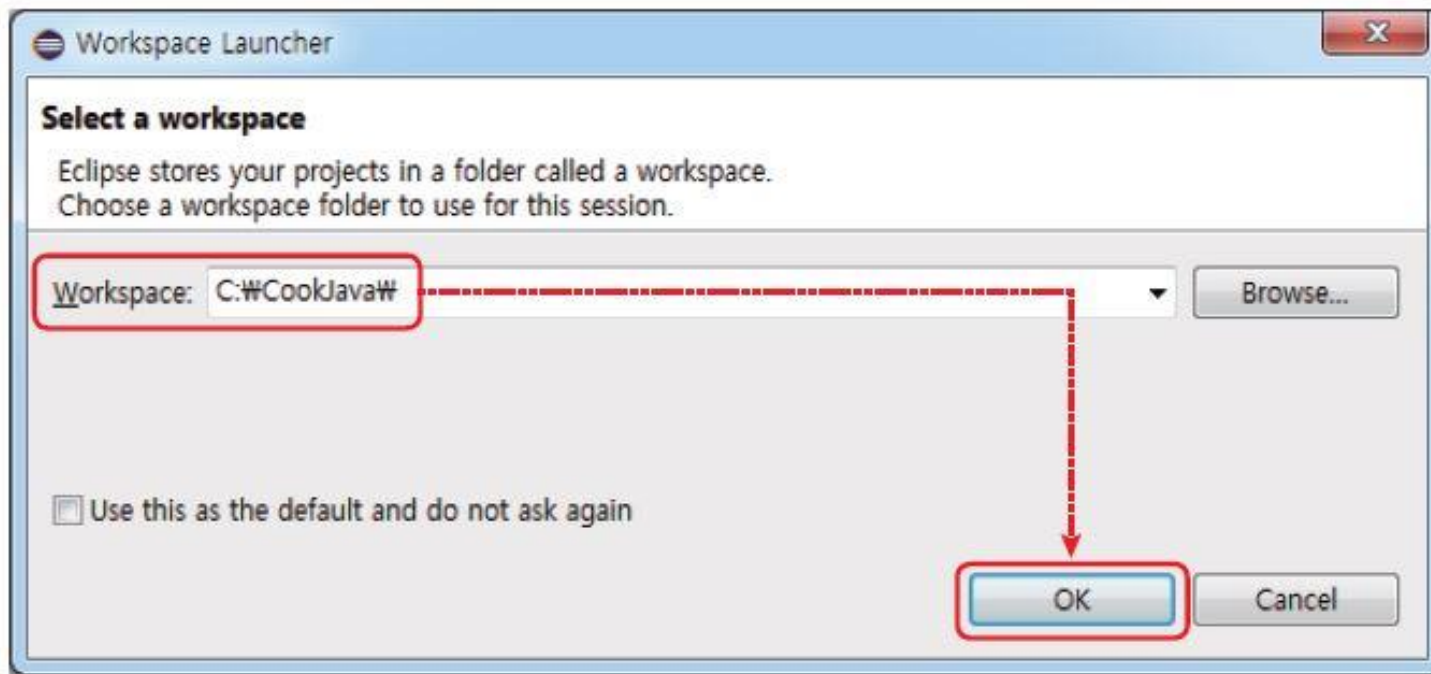
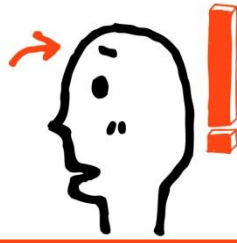


그림 2-6 이클립스 워크스페이스 지정



실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 워크스페이스(workspace)

프로젝트 폴더가 저장

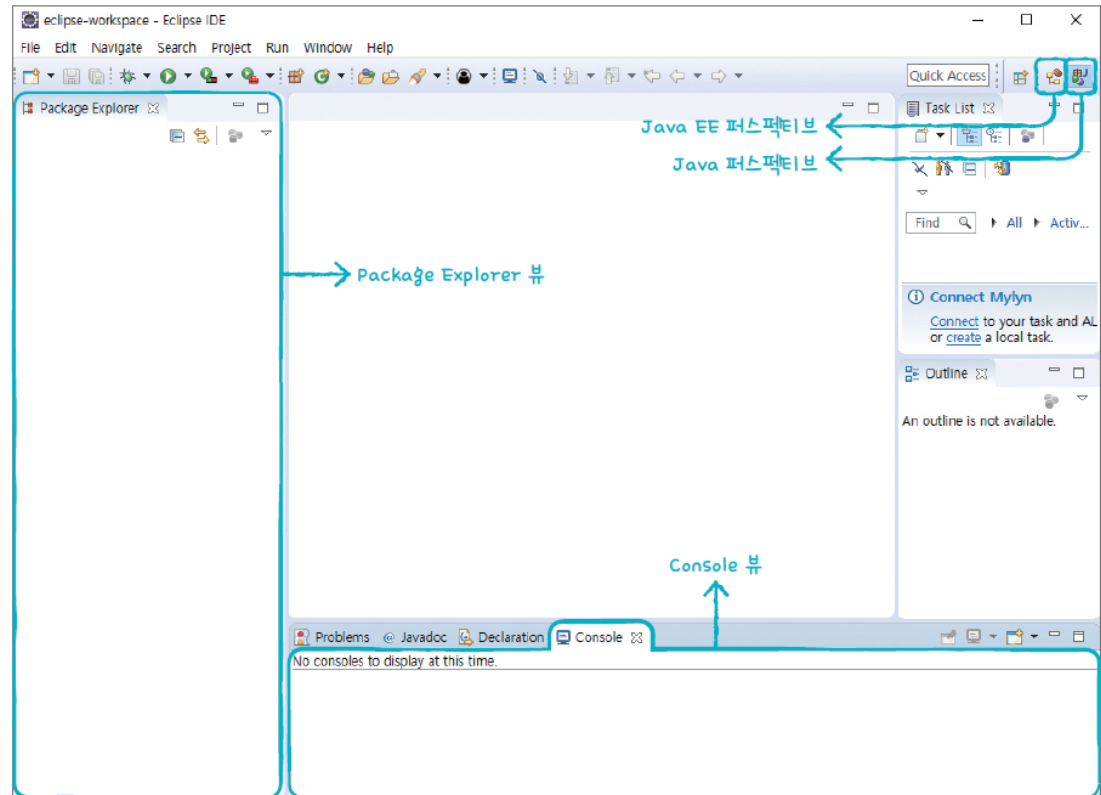
개발 환경 정보와 관련된 메타 데이터가 저장된 폴더(.metadata)가 저장

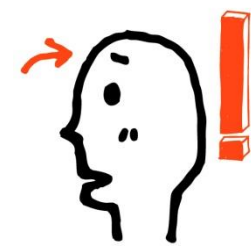
▶ 퍼스펙티브(perspective)

프로젝트를 개발할 때 유용하게 사용할 수 있는 뷰View들을 미리 묶어 이름을 붙여놓은 것

▶ 뷰(view)

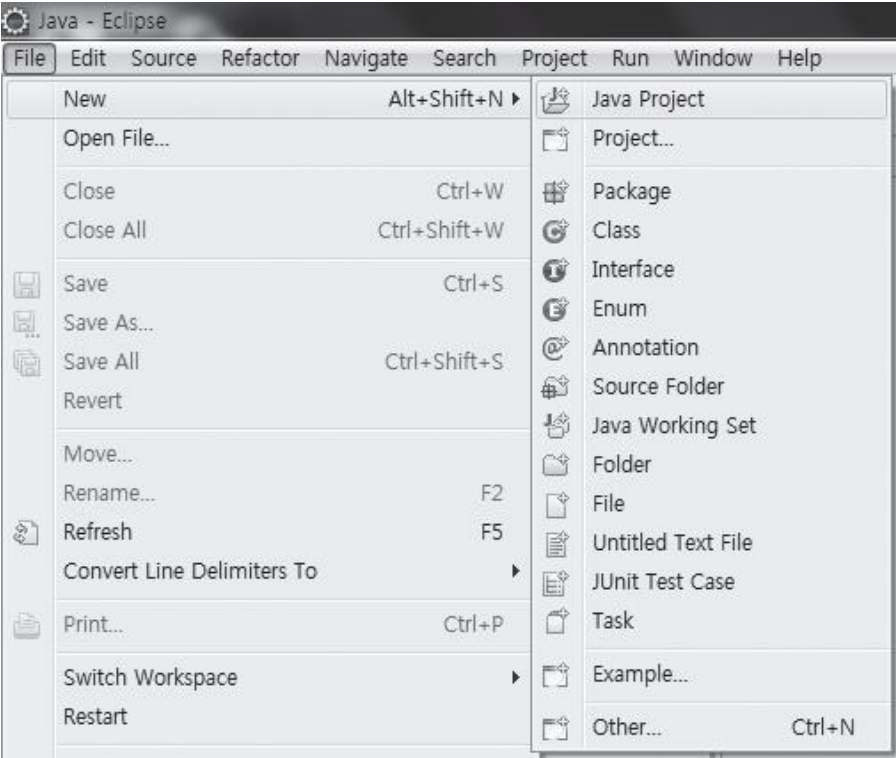
이클립스 내부에서 사용되는 작은 창





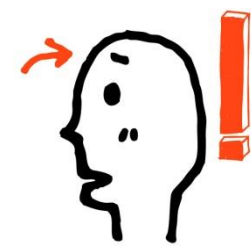
실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 프로젝트 만들기



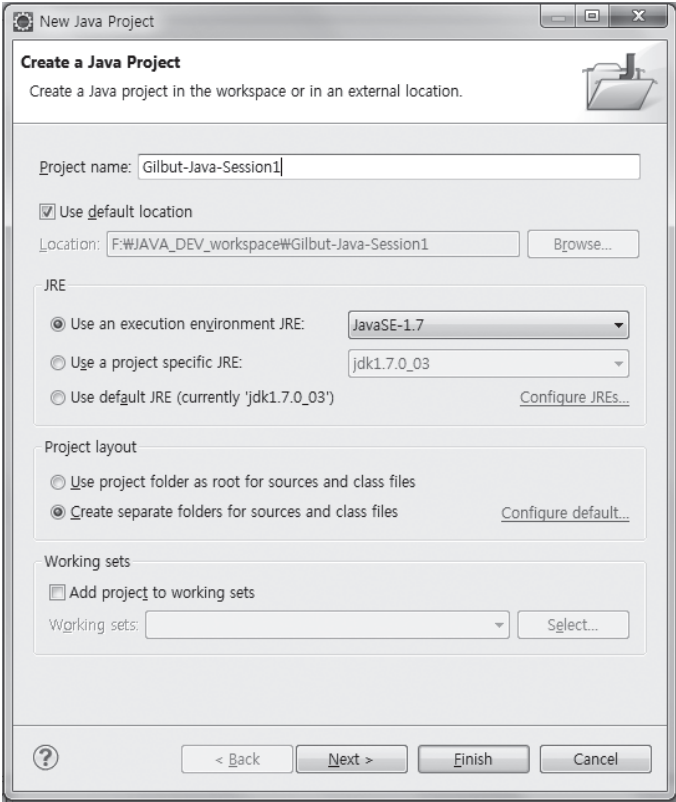
1 Package Explorer 뷰에서
마우스 오른쪽 버튼을 클릭

New → Java Project을 선택



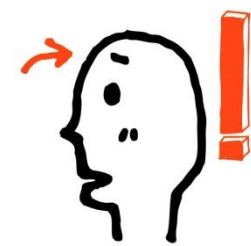
실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 프로젝트 만들기



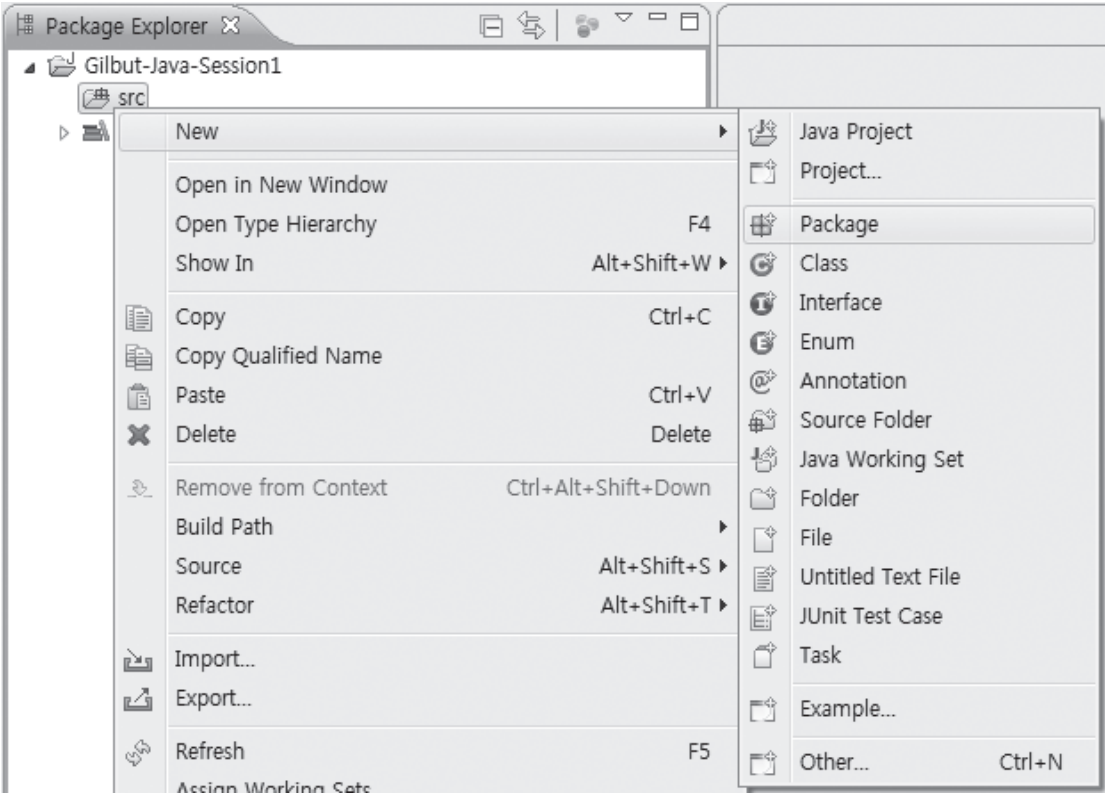
② Project name 항목에
Gilbut-Java-Session1라 입력

<Finish> 버튼을 클릭해서
새로운 자바 프로젝트를 생성



실무에 적합한 개발 도구, 이클립스와 친해지자

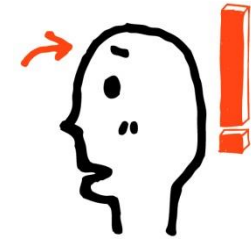
▶ 프로젝트 만들기



3 프로젝트를 새로 생성하면
하위 메뉴에 기본 적으로
src 이름의 소스 폴더가 생성됨

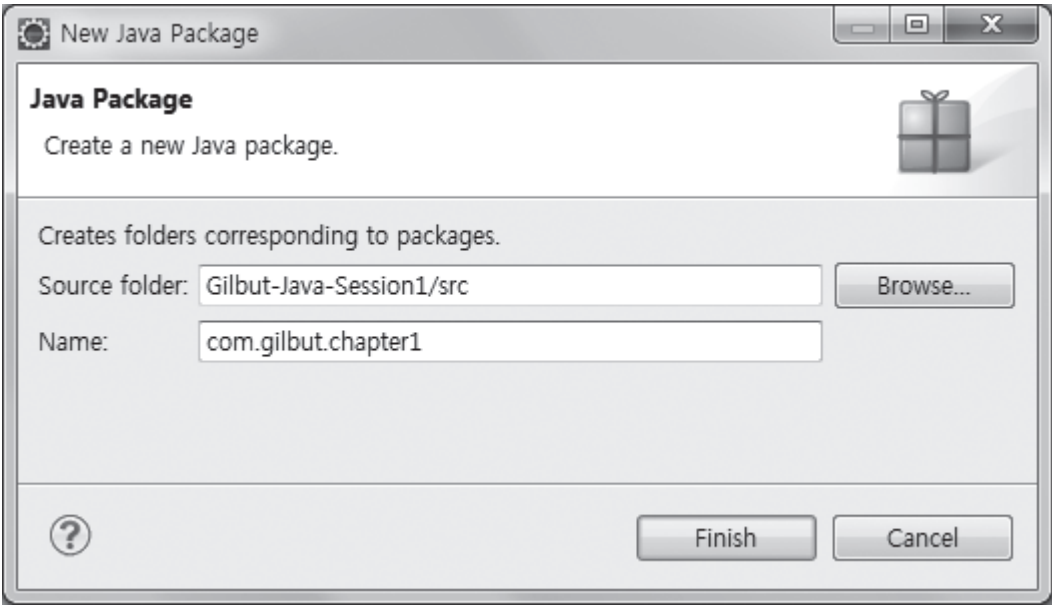
기본 소스 폴더에
마우스 오른쪽 버튼을 클릭

new → Package를 선택



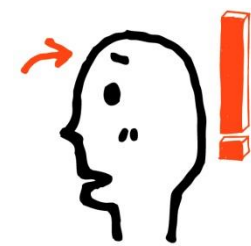
실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 프로젝트 만들기



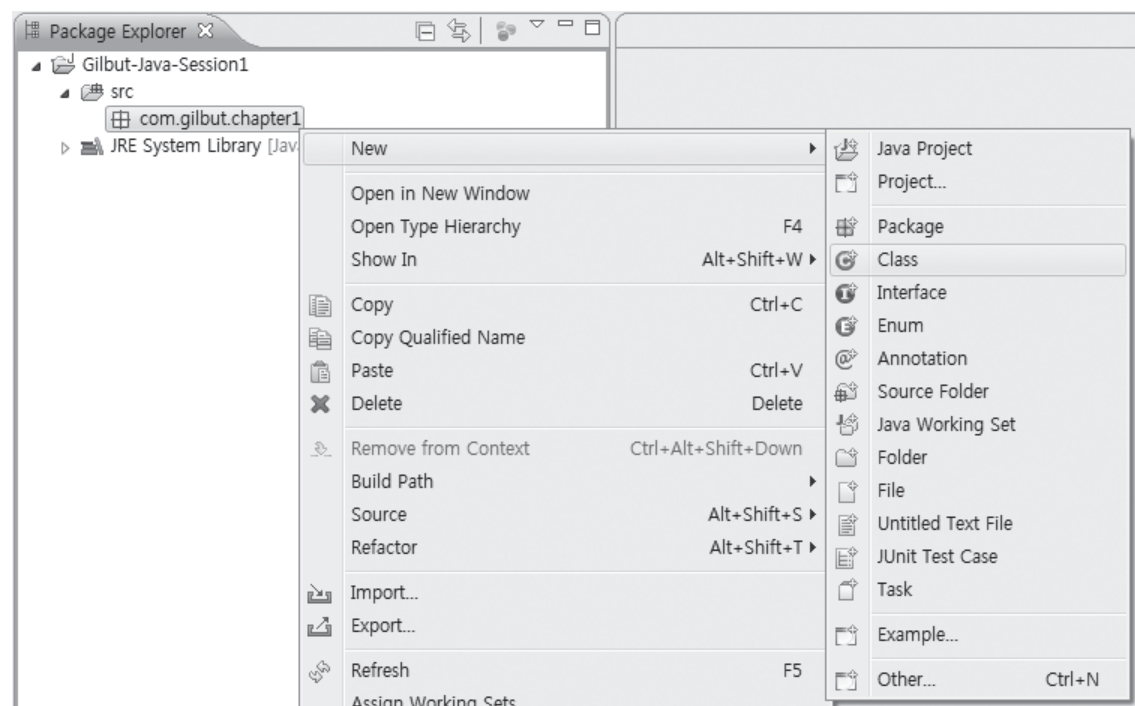
- ④ Name 항목에
com.gilbut.chapter1를 입력
- <Finish> 버튼을 클릭

▶ 패키지 이름은 보통 해당 프로젝트의 도메인 역순으로 이름을 설정



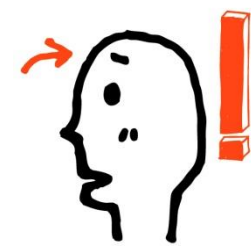
실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 프로젝트 만들기



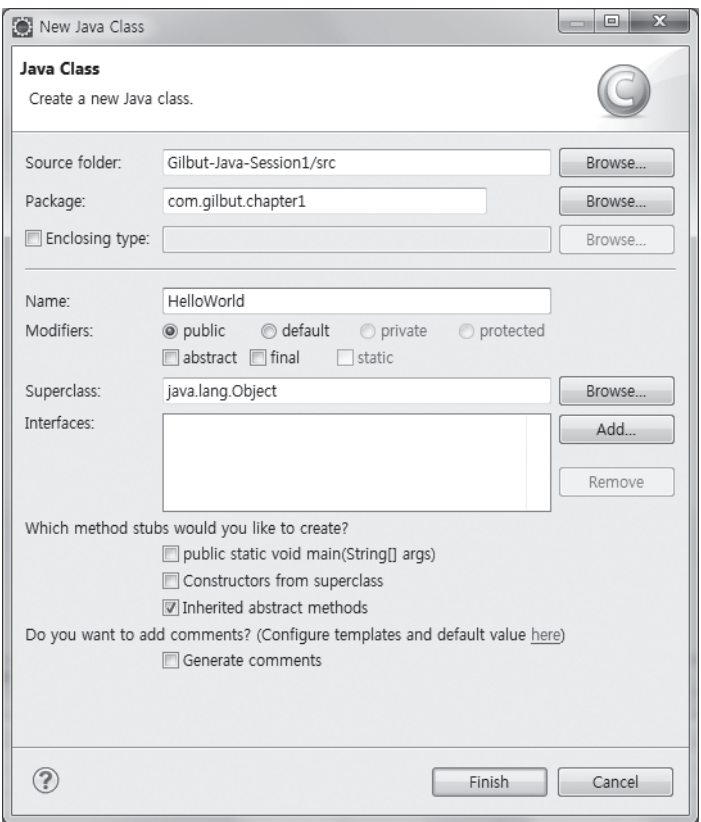
5 새로 생성한 패키지에서
마우스 오른쪽 버튼 클릭

New → Class를 클릭하여
자바 소스 파일을 만듦



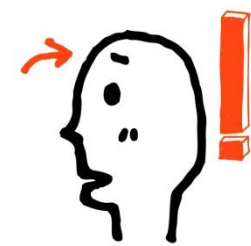
실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 프로젝트 만들기



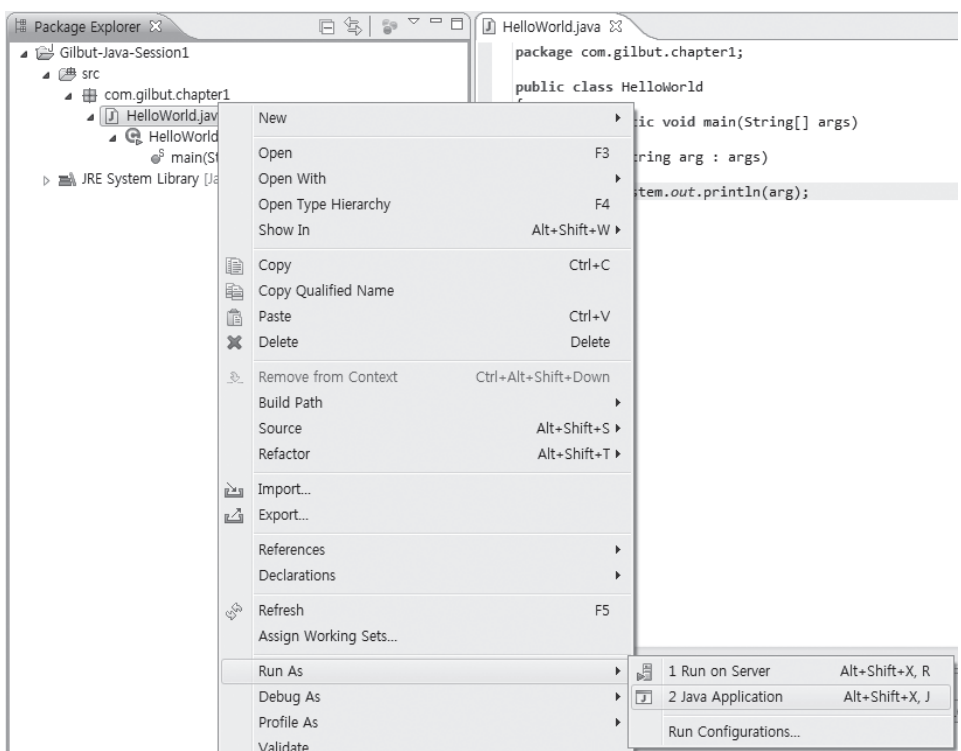
⑥ Name 항목에 클래스 이름인 'HelloWorld'라 입력

<Finish> 버튼을 클릭



실무에 적합한 개발 도구, 이클립스와 친해지자

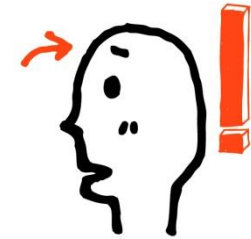
▶ 프로젝트 만들기



7 자바 퍼스펙티브 왼쪽에 위치한 Package Explorer 뷰에 'HelloWorld.java'를 마우스 오른쪽 버튼을 클릭

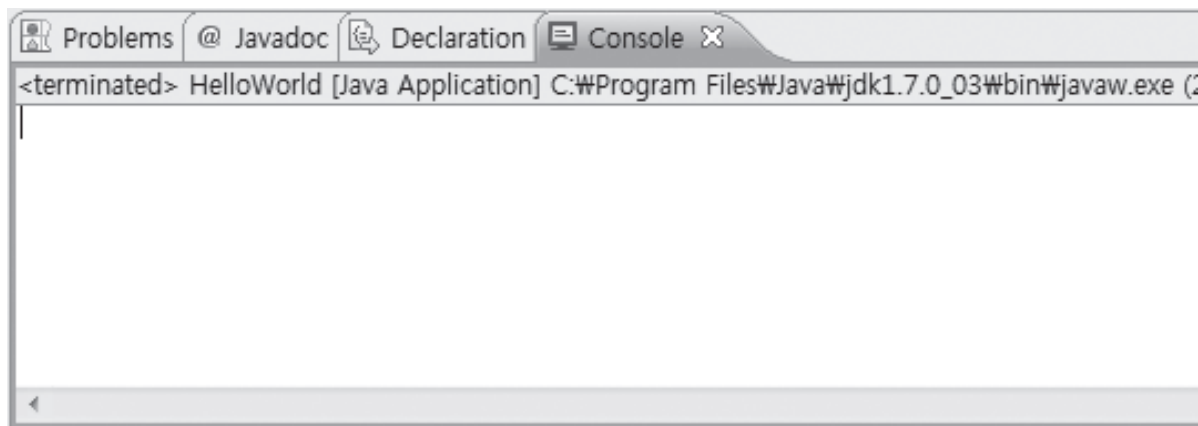
Run As → Java Application을 선택

이클립스를 처음 실행할 경우 저장을 묻는 대화 상자가 나타남. 'Always save resources before launching'에 체크한 뒤 <OK> 버튼을 클릭

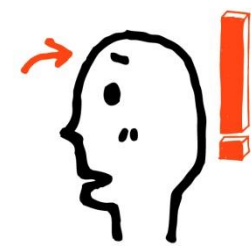


실무에 적합한 개발 도구, 이클립스와 친해지자

▶ 프로젝트 만들기



- ⑧ 자동으로 컴파일된 HelloWorld 클래스가 실행되고 활성화된 Console창에 결과가 출력되지만 인자(argument)를 입력하지 않았으므로 Console 창에 아무것도 볼 수 없음 (Console 뷰에 결과가 나타나지 않았지만 실제로 HelloWorld.class 파일은 실행됨)



실무에 적합한 개발 도구, 이클립스와 친해지자

- 개발이 편리하도록 이클립스 설정 변경하기
- ① 행 번호 보이게 하기 : 앞으로 코딩할 소스의 행 번호가 보이도록 설정. Eclipse 메뉴의 [Window]-[Preferences]를 선택, 왼쪽 트리에서 [General]-[Editors]-[Text Editors] 선택, 오른쪽의 'Show line numbers'가 체크되어 있는지 확인. 체크되어 있지 않으면 체크

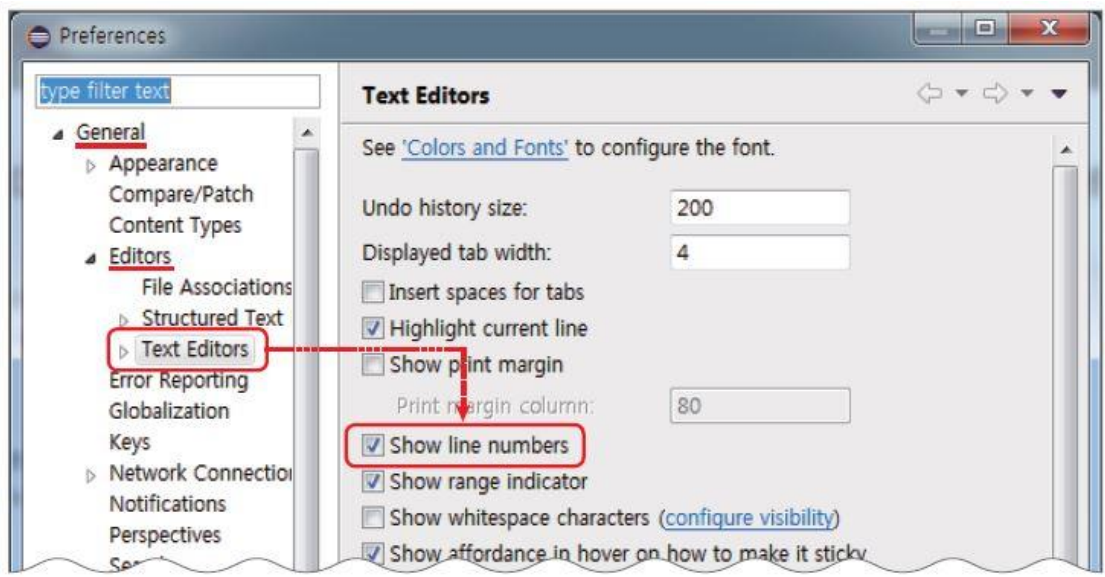
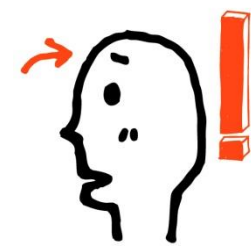


그림 2-8 이클립스 설정 : 행 번호 보이게 하기



실무에 적합한 개발 도구, 이클립스와 친해지자

② 폰트 변경하기 : 왼쪽 트리의 [General]-[Appearance]-[Colors and Fonts] 선택. 오른쪽 트리의 [Basic]-[Text Font] 선택, [Edit] 클릭, 원하는 글꼴, 크기 등 선택, [확인] 클릭. [OK]를 클릭하여 'Preferences' 창 종료

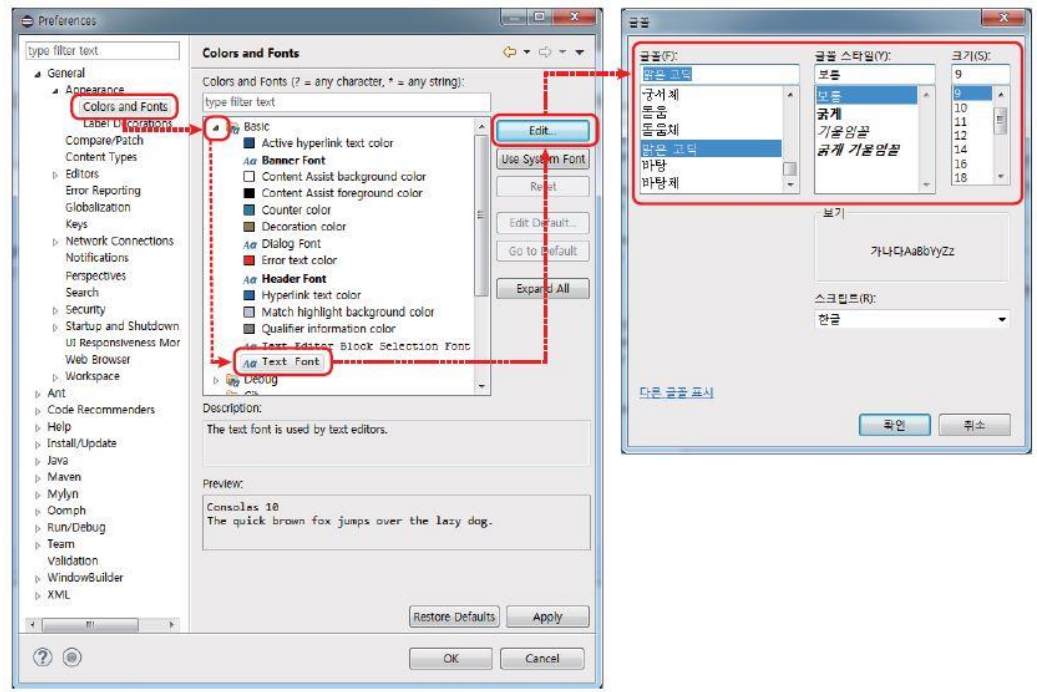
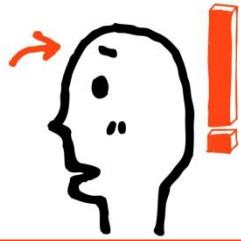


그림 2-9 이클립스 설정: 폰트 변경하기

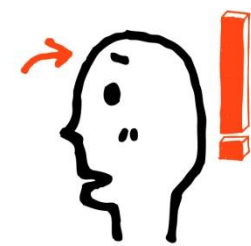


실무에 적합한 개발 도구, 이클립스와 친해지자

▶ System.out.println의 다양한 활용: 프로그램의 수행된 결과를 출력

```
class SystemOutPrintln
{
    public static void main(String[] args)
    {
        System.out.println(7);
        System.out.println(3.15);
        System.out.println("3+5=" + 8);
        System.out.println(3.15 + "는 실수입니다.");
        System.out.println("3+5" + "의 연산결과는 8입니다.");
        System.out.println(3+5);
    }
}
```

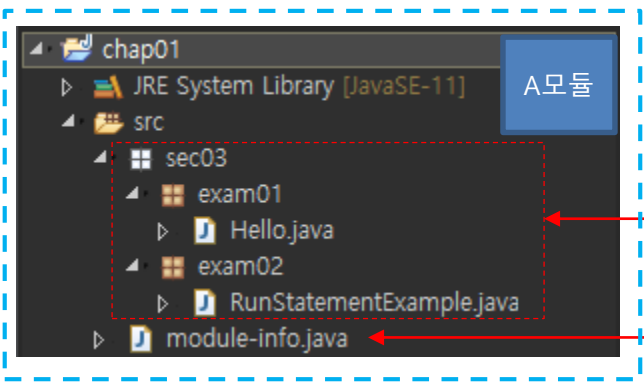
```
7
3.15
3+5=8
3.15는 실수입니다.
3+5의 연산결과는 8입니다.
8
```



모듈 기술자(Java 11 버전 이후)

▶ 모듈(Module)이란

- 외부에서 재사용할 수 있는 패키지들을 묶은 것
- 이클립스의 프로젝트는 하나의 모듈을 개발하는 것



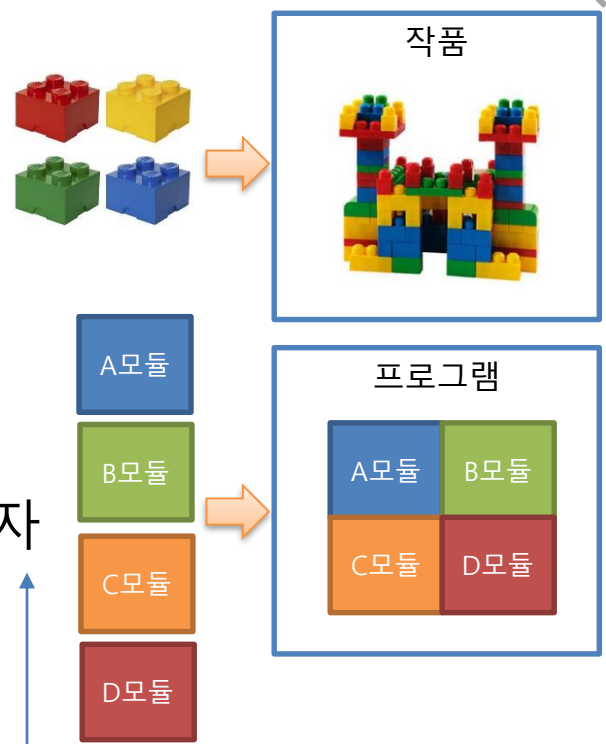
모듈

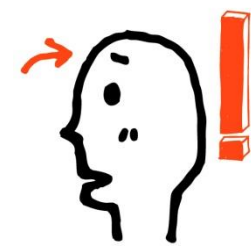
패키지

모듈 기술자

▶ 모듈 기술자(module-info.java)

- 해당 **모듈의 이름**이 무엇인지,
- 해당 모듈이 **의존하는 모듈**이 무엇인지
- 해당 모듈을 외부에서 어떻게 사용할 수 있는지





모듈 기술자(Java 11 버전 이후)

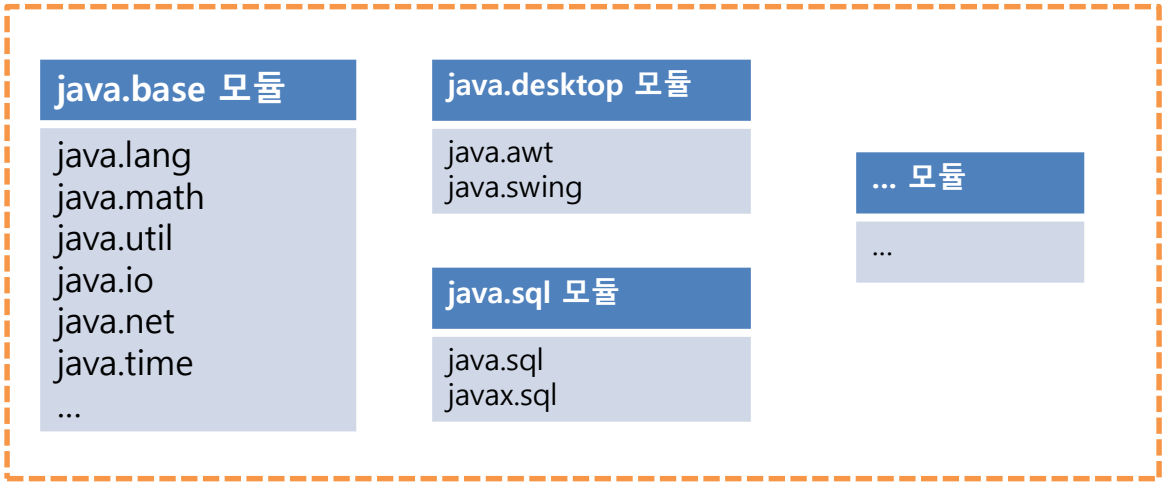
▶ 의존하는 모듈

- 해당 모듈(프로젝트)가 실행하기 위해서 필요한 외부 모듈을 말함

Java 11 이후 버전 java.se 모듈

chap01 모듈(프로젝트)

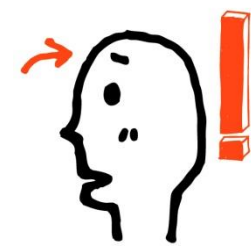
sec03.exam01
sec03.exam02



- 기본적으로 **java.base** 모듈만 사용 가능하므로 다른 모듈을 사용할 경우 의존 모듈로 등록해야 함

```
module-info.java x
1 module chap01 {
2     requires java.desktop;
3     requires java.sql;
4 }
```

```
*module-info.java x
1 module chap01 {
2     requires java.se;
3 }
```

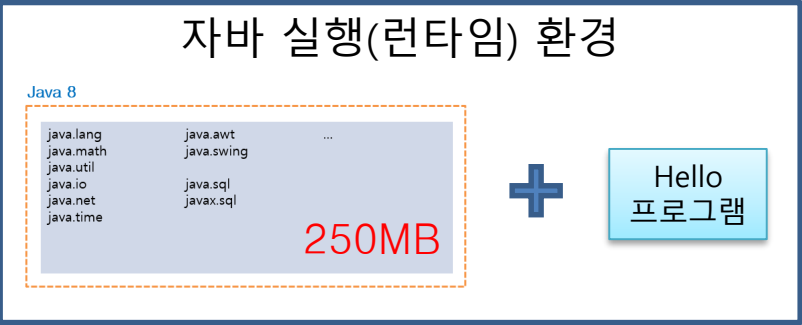
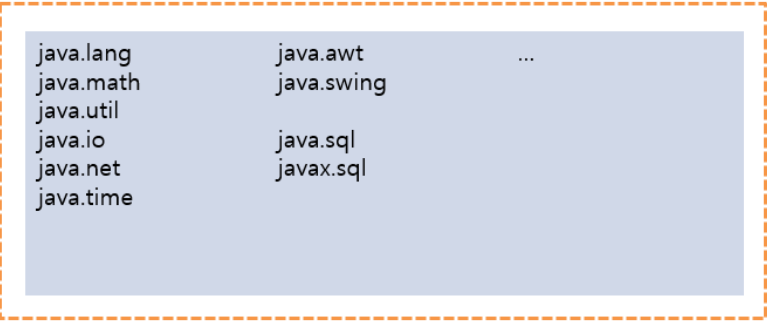


모듈화가 필요한 이유

▶ 모듈화가 필요한 이유

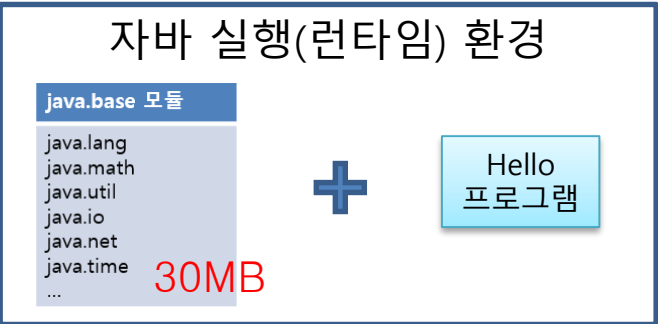
- 패키지 보안
- 작은 최적의 런타임 이미지를 만들기 위해

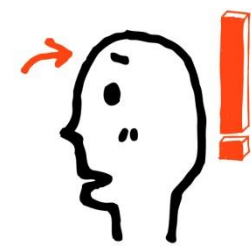
Java 8



Java 11 이후 버전

java.se 모듈





컴파일에서 제외되는 주석

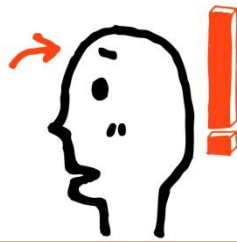
▶ 주석

- ▶ 개발자가 만든 소스 코드 중에 컴파일 시 컴파일러에 의해 무시되는 부분 의미
- ▶ 소스 코드 컴파일 시 컴파일러는 소스 코드에 있는 주석 부분은 컴파일 무시
- ▶ 소스 코드에 대한 설명 부분이 필요한 경우 사용
- ▶ 현재 소스 코드 중 일부분을 수행되지 않도록 해줄 때 사용

▶ 주석의 활용

- ▶ 소스 코드 분석 시 소스 코드를 통해 이해한 내용을 주석으로 정리
- ▶ 개발한 이후 나중에 소스 코드에 대한 내용을 쉽게 알아 볼 수 있도록 주석으로 정리

구분	주석 기호	설명
라인 주석	// ...	//부터 라인 끝까지 주석으로 처리합니다.
범위 주석	/* ... */	/*와 */ 사이에 있는 내용은 모두 주석으로 처리합니다.
도큐먼트 주석	/** ... */	/**와 */ 사이에 있는 내용은 모두 주석으로 처리합니다. 주로 javadoc 명령어로 API 도큐먼트를 생성하는 데 사용합니다.



컴파일에서 제외되는 주석

▶ 행 단위의 주석

```
// 파일이름 : LineComment.java  
// 작성자 : 홍길동  
// 작성일 : 2012년 9월 25일  
// 작성이유 : System.out.println 메소드 기능 테스트
```

```
class SystemOutPrintln  
{  
    public static void main(String[] args)  
    {  
        System.out.println(7);    // 정수의 출력  
        System.out.println(3.15);  
        System.out.println("3+5=" + 8);  
        System.out.println(3.15 + "는 실수입니다.");  
        System.out.println("3+5" + "의 연산결과는 8입니다.");  
        System.out.println(3+5);    // 덧셈 결과 출력  
    }  
}
```